Verification of the Non-linear Quadrotor Simulator

Validating the Implemented Code with the Derived Equations of Motion

Henry Rochester
Supervisor: Dr Malcolm Hillebrand

April 6, 2026

Introduction

We recall that the objective of this project is to develop and compare multiple control methods for a non-linear quadrotor system, namely a PID controller, an LQR controller, and a reinforcement learning-based (PPO) controller. These controllers are evaluated within a simulated environment derived from first-principles modelling of the quadrotor's equations of motion.

Before implementing and analysing the performance of these controllers, it is essential to first verify that the simulator accurately reflects the underlying physical system. In particular, we require that the numerical implementation of the equations of motion corresponds correctly to the dynamics derived from first principles.

To establish this, we conduct a series of open-loop experiments, in which control inputs are applied independently of the system's state. This allows us to isolate and validate the simulator behaviour without the influence of a closed-loop, feedback control system. The experiments include a hover equilibrium test, vertical thrust response, periodic thrust input, pure rotational motions (roll, pitch, and yaw), and a free-fall test. Representative trajectories are analysed and compared against the expected behaviour predicted by the governing equations.

We note that this verification step is crucial, as it ensures that the simulator is mathematically and physically consistent. Without such validation, any conclusions drawn from controller performance would be unreliable, as discrepancies could arise from modelling errors rather than control design.

Having established confidence in the simulator, we proceed to implement and evaluate the different control strategies under consistent conditions.

State-Space Formulation of the Quadrotor Model

Recall that the state vector of the quadrotor system is given by:

$$\vec{x} = \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \\ \phi \\ \theta \\ \psi \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \in \mathbb{R}^{12},$$

where (x, y, z) denote position, (v_x, v_y, v_z) denote linear velocity, (ϕ, θ, ψ) denote the roll, pitch, and yaw (Euler) angles respectively, and $(\omega_x, \omega_y, \omega_z)$ denote the angular velocities in the body frame.

We model the system in state-space form as a non-linear dynamical system given by:

$$\dot{\vec{x}} = f(\vec{x}, \vec{u}),$$

where $\vec{u} \in \mathbb{R}^4$ is the control input vector defined by:

$$\vec{u} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix},$$

with u_i representing the thrust generated by rotor i, where u_1 corresponds to the top-right rotor, u_2 to the top-left rotor, u_3 to the bottom-left rotor, and u_4 to the bottom-right rotor.

The simulator numerically integrates this twelve-dimensional non-linear system forward in time. To clearly establish the correspondence between the physical model and its

numerical implementation, we now decompose the system $\dot{\vec{x}} = f(\vec{x}, \vec{u})$ into its constituent components as follows.

Translational and Rotational Components

Translational Motion

The translational kinematics are given by:

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad \dot{z} = v_z.$$

From Newton's second law, the translational dynamics satisfy:

$$m\dot{\vec{v}} = m\vec{g} + R(\phi, \theta, \psi)\,\vec{F}_T,$$

where $\vec{g} = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix}$ is the gravitational acceleration vector, $R(\phi, \theta, \psi)$ is the rotation matrix mapping vectors from the drone's body frame to the world's inertial reference frame, and $\vec{F}_T$ is the thrust vector expressed in the body frame.

Dividing through by m, we obtain:

$$\dot{\vec{v}} = \vec{g} + \frac{1}{m} R(\phi, \theta, \psi)\,\vec{F}_T.$$

Since thrust acts along the body-frame z-axis, the thrust vector is given by:

$$\vec{F}_T = \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix},$$

where the total thrust is

$$T = u_1 + u_2 + u_3 + u_4.$$

Thus, the translational dynamics can be written as:

$$\dot{\vec{v}} = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} + \frac{1}{m} R(\phi, \theta, \psi) \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix}.$$

Rotational Motion

For notational simplicity, we define the Euler angle vector:

$$\vec{\eta} = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}.$$

From the full derivation of the non-linear quadrotor dynamics, the orientation of the drone is described using a Z-Y-X (yaw-pitch-roll) rotation sequence. As a result, the time derivative of the Euler angles is related to the body-frame angular velocity $\vec{\omega} = (\omega_x, \omega_y, \omega_z)$ by:

$$\dot{\vec{\eta}} = \begin{bmatrix} 1 & 0 & -\sin\theta \\ 0 & \cos\phi & \sin\phi\cos\theta \\ 0 & -\sin\phi & \cos\phi\cos\theta \end{bmatrix}^{-1} \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}.$$

From the Newton-Euler rigid body equation,

$$\mathbf{I}\dot{\vec{\omega}} + \vec{\omega} \times (\mathbf{I}\vec{\omega}) = \vec{\tau},$$

and noting that for a symmetric quadrotor the inertia matrix is diagonal,

$$\mathbf{I} = \begin{bmatrix} I_x & 0 & 0 \\ 0 & I_y & 0 \\ 0 & 0 & I_z \end{bmatrix},$$

we obtain the rotational dynamics:

$$\begin{aligned} \dot{\omega}_x &= \frac{1}{I_x}\left(\tau_x - (I_z - I_y)\,\omega_y\omega_z\right), \\ \dot{\omega}_y &= \frac{1}{I_y}\left(\tau_y - (I_x - I_z)\,\omega_z\omega_x\right), \\ \dot{\omega}_z &= \frac{1}{I_z}\left(\tau_z - (I_y - I_x)\,\omega_x\omega_y\right). \end{aligned}$$

For an X-configuration quadrotor, the control torques are given by:

$$\begin{aligned} \tau_x &= \frac{l}{\sqrt{2}}(u_2 + u_3 - u_1 - u_4), \\ \tau_y &= \frac{l}{\sqrt{2}}(u_1 + u_2 - u_3 - u_4), \\ \tau_z &= k_d(u_1 - u_2 + u_3 - u_4), \end{aligned}$$

where k_d denotes the torque-to-thrust ratio.

Full System of Equations of Motion

Collecting the translational and rotational components, the full non-linear quadrotor dynamics are given by the following system of 12 equations of motion:

$$
\begin{aligned}
\dot{x} &= v_x, \\
\dot{y} &= v_y, \\
\dot{z} &= v_z, \\
\dot{v}_x &= \frac{T}{m}\left(\cos\phi\sin\theta\cos\psi + \sin\phi\sin\psi\right), \\
\dot{v}_y &= \frac{T}{m}\left(\cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi\right), \\
\dot{v}_z &= -g + \frac{T}{m}\left(\cos\phi\cos\theta\right), \\
\dot{\phi} &= \omega_x + \sin\phi\tan\theta\,\omega_y + \cos\phi\tan\theta\,\omega_z, \\
\dot{\theta} &= \cos\phi\,\omega_y - \sin\phi\,\omega_z, \\
\dot{\psi} &= \frac{\sin\phi}{\cos\theta}\,\omega_y + \frac{\cos\phi}{\cos\theta}\,\omega_z, \\
\dot{\omega}_x &= \frac{1}{I_x}\left(\tau_x - \left(I_z - I_y\right)\omega_y\omega_z\right), \\
\dot{\omega}_y &= \frac{1}{I_y}\left(\tau_y - \left(I_x - I_z\right)\omega_z\omega_x\right), \\
\dot{\omega}_z &= \frac{1}{I_z}\left(\tau_z - \left(I_y - I_x\right)\omega_x\omega_y\right),
\end{aligned}
$$

Implementation of the Equations of Motion in the Simulator

We now explain how the mathematical model derived above is implemented within the simulator.

The simulator stores the full state vector given by:

$$(x, y, z, v_x, v_y, v_z, \phi, \theta, \psi, \omega_x, \omega_y, \omega_z),$$

which matches precisely the twelve-dimensional state vector derived above.

The system dynamics are implemented within the `dynamics.py` module as a function that evaluates the right-hand side of the state-space model

$$\dot{\vec{x}} = f(\vec{x}, \vec{u}),$$

where, given the current state $\vec{x}$ and control input $\vec{u}$, the function returns the time derivative $\dot{\vec{x}}$. Each component of this function corresponds directly to the derived equations of motion above, with the state derivative $\dot{\vec{x}}$ decomposed into derivatives of position, linear velocity, Euler angles, and angular velocity.

To evolve the system forward in time, the simulator employs a numerical integration scheme. In `simulator.py`, a fourth-order Runge-Kutta (RK4) method is used to approximate the solution of the differential equation over a small time step $\Delta t = 0.01$. Given the current state and control input, the RK4 method evaluates the dynamics function, 4 times, within this small interval to compute the next state $\vec{x}_{t+1}$, providing a more accurate and stable discretisation of the non-linear system, compared to the Euler forward method.

Within `hover_env.py`, this integrator is called at each time step of the environment. For every control input applied, the simulator advances the state using the RK4 method, thereby producing a discrete-time trajectory that approximates the underlying continuous-time dynamics over an interval of 5 seconds (500 time steps, with each time step being 0.01s). This enables us to accurately capture the evolution of the drone's state as it advances forward in time.

Verification Strategy

To verify that the simulator accurately implements the equations of motion, we conduct a series of open-loop experiments and compare the simulated behaviour with the expected response derived from the mathematical model.

The verification procedure is as follows:

1. select a physically meaningful control input that induces a clear and interpretable change in the system state;
2. predict the qualitative or quantitative system response using the derived equations of motion;
3. simulate the system numerically using the `hover_env.py` module;
4. compare the simulated response with the predicted behaviour.

Simulation Experiments

Experiment 1: Hover Equilibrium Test

We begin with the simplest possible test: hover equilibrium.

The drone is initialised at a fixed position, level orientation, and with total constant thrust equal to its weight. We further assume that there are no stochastic disturbances (i.e. no wind nor sensor noise). Under these conditions, we expect zero acceleration and hence no change in position or orientation.

More precisely, we impose the conditions:

$$T = mg, \qquad \tau_x = \tau_y = \tau_z = 0, \qquad x = 0,\ y = 0,\ z = 5,$$

under which the model predicts that the quadrotor remains in a hover state at the specified position.

This follows directly from the translational dynamics:

$$\dot{\vec{v}} = \vec{g} + \frac{1}{m} R(\phi, \theta, \psi) \vec{F}_T,$$

which reduces to zero when the thrust exactly balances gravity and the drone is level. Furthermore, zero applied torques imply that the angular acceleration is zero, and hence both the angular velocity and orientation remain constant. Thus, we expect:

- $z(t)$ remains constant;
- $v_z(t) = 0$ for all t;
- $\phi(t), \theta(t), \psi(t)$ remain constant;
- $\omega_x(t), \omega_y(t), \omega_z(t) = 0$ for all t.

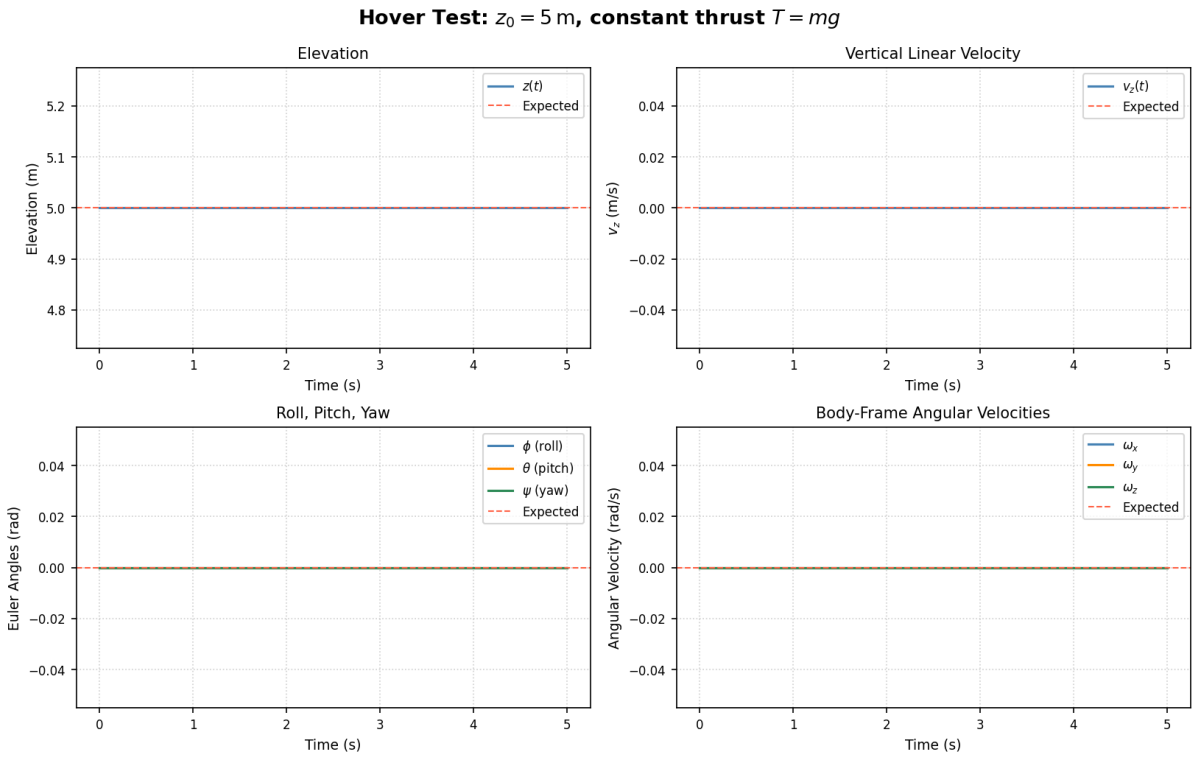

Figure 1: Hover equilibrium test under $T = mg$ and zero applied torques.

We observe from the figure that, as expected, the elevation (z-position) of the drone remains constant over 500 time steps, corresponding to 5 seconds of simulated time.

Furthermore, the vertical velocity satisfies $v_z(t) = 0$ for all t, indicating no motion in the vertical direction. The Euler angles $\phi(t), \theta(t), \psi(t)$ remain constant at 0 radians, confirming that the drone maintains a level orientation throughout the simulation.

Additionally, the angular velocities $\omega_x(t), \omega_y(t), \omega_z(t)$ remain identically zero, indicating the absence of rotational motion.

These results are in complete agreement with the theoretical prediction for hover equilibrium under $T = mg$ and zero applied torques.

Experiment 2: Vertical Ascent and Descent Test

We now apply a constant thrust offset from the hover thrust in the vertical direction. Specifically, we define:

$$T = mg + \Delta T, \qquad \tau_x = \tau_y = \tau_z = 0,$$

where ΔT represents the deviation of the applied thrust from the hover thrust required to balance the weight of the quadrotor.

As in the previous experiment, the drone remains level and balanced, implying zero applied torques and hence no rotational motion. Therefore, the motion is restricted entirely to the vertical (z) direction.

It follows from the translational dynamics that:

$$m\ddot{z} = -mg + T.$$

Substituting $T = mg + \Delta T$, we obtain:

$$m\ddot{z} = \Delta T, \qquad \text{and hence} \qquad \ddot{z} = \frac{\Delta T}{m}.$$

In this experiment, however, we prescribe the thrust as a fixed proportion of the hover thrust, taking:

$$T_{\text{climb}} = 1.5\, mg, \qquad T_{\text{descent}} = 0.5\, mg.$$

This corresponds to a thrust offset of the form $\Delta T = \alpha mg$ for some constant α, and hence the resulting vertical acceleration becomes:

$$\ddot{z} = \frac{\Delta T}{m} = \alpha g,$$

which is constant and independent of the mass of the drone.

Thus, the model predicts constant vertical acceleration. In particular:

- if $\Delta T > 0$, the drone undergoes constant upward acceleration (ascent);
- if $\Delta T < 0$, the drone undergoes constant downward acceleration (descent).

Integrating once, the vertical velocity $v_z(t)$ is expected to vary linearly in time. Integrating again, the elevation $z(t)$ is expected to follow a quadratic trajectory.

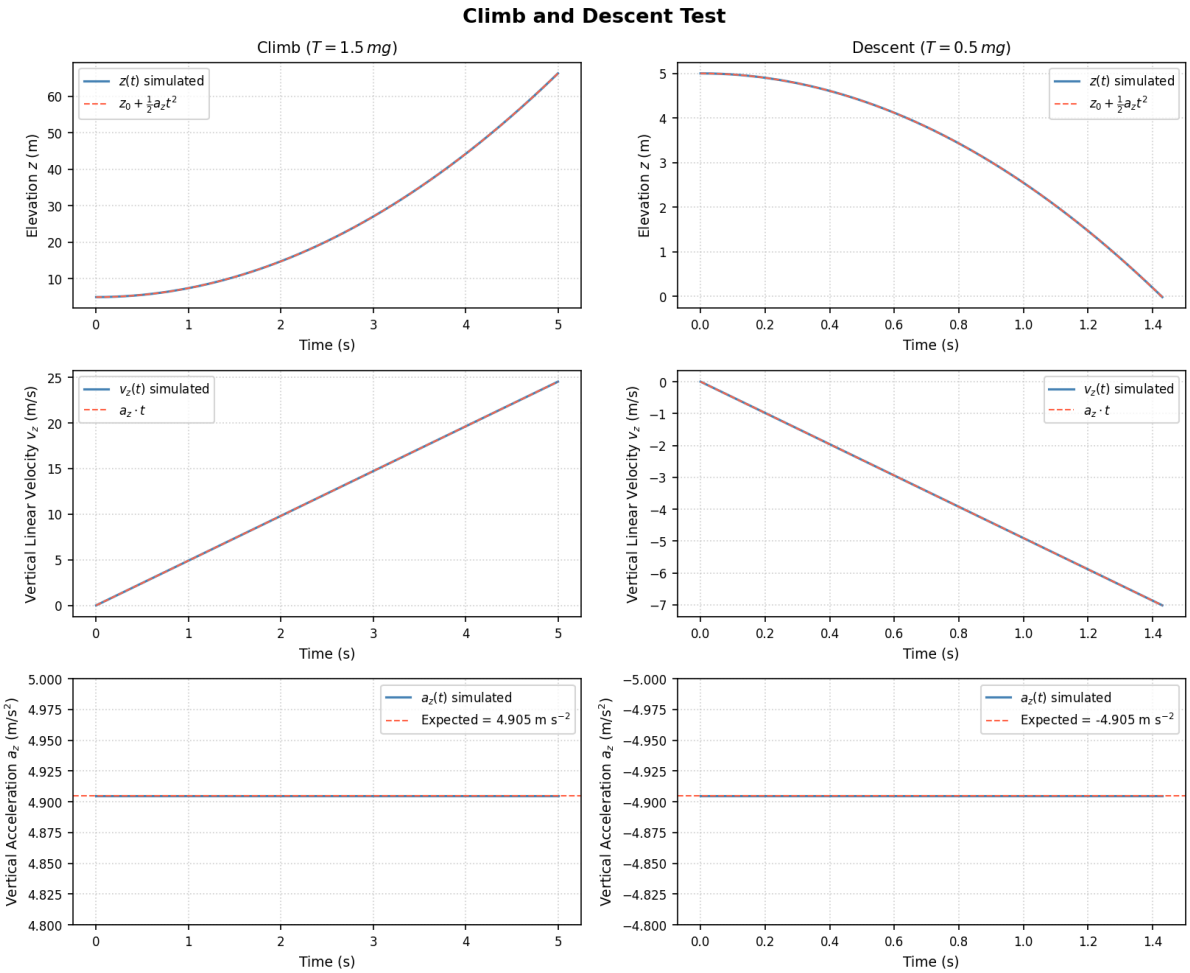

Figure 2: Vertical response under a constant thrust offset above and below hover thrust.

As expected, we observe a constant positive vertical acceleration under the climbing thrust, and a constant negative vertical acceleration under the descent thrust. This is consistent with the fact that the applied thrust is a fixed proportion above and below the hover thrust, respectively.

Examining the vertical velocity, we obtain a linear relationship with respect to time in both cases. This aligns with the mathematical result that integrating a constant acceleration yields a linear function. Specifically, we observe a linear increase in vertical velocity during ascent, and a linear decrease during descent.

Finally, considering the elevation, we observe a quadratic trajectory in both cases. This follows directly from the fact that integrating a linear function yields a quadratic function. For the ascent case, the elevation increases quadratically, and evaluating the expression:

$$z(t) = z_0 + \frac{1}{2}a_z t^2$$

at $t = 5$ with $a_z = 4.905$ yield:

$$z(5) = 5 + \tfrac{1}{2}(4.905)(5^2) \approx 66.31,$$

which matches the simulated result.

Similarly, for the descent case, we observe a quadratic decrease in elevation. Furthermore, the simulation correctly terminates when the drone reaches the ground. This is enforced via the `_is_terminated()` condition within `hover_env.py`, and is reflected in the plot where the drone reaches ground level at approximately $t \approx 1.4$ seconds. This not only validates the physical behaviour of the system, but also ensures computational efficiency by terminating the episode once the terminal condition is met.

Experiment 3: Periodic Thrust Input

Next, we consider a time-varying periodic thrust input of the form:

$$T(t) = mg + A\sin(\nu t),$$

with zero applied torque (i.e. $\tau_x = \tau_y = \tau_z = 0$). Here, A denotes the amplitude of the thrust variation, and ν is the angular frequency, determining how rapidly the thrust input is adjusted over time.

Assuming the drone remains approximately level, the vertical dynamics simplify to:

$$m\ddot{z} = -mg + mg + A\sin(\nu t) = A\sin(\nu t),$$

and hence, we obtain:

$$\ddot{z} = \frac{A}{m}\sin(\nu t).$$

From this, we expect the vertical acceleration to vary sinusoidally, directly mirroring the oscillatory thrust input about the hover thrust mg. Integrating once, and applying the initial condition $\dot{z}(0) = 0$ (corresponding to initial hover), we obtain

$$\dot{z}(t) = \frac{A}{m\nu}\big(1 - \cos(\nu t)\big).$$

Since $|\cos(\nu t)| \leq 1$ and $A, m, \nu > 0$, it follows that $\dot{z}(t) \geq 0$ for all t. Thus, the vertical velocity is oscillatory but remains non-negative, with minimum value 0.

Integrating once more, and applying the initial condition $z(0) = z_0$, we obtain:

$$z(t) = z_0 + \frac{A}{m\nu}t - \frac{A}{m\nu^2}\sin(\nu t).$$

Hence, the elevation is expected to exhibit a smooth oscillatory variation with an overall upward trajectory, as the vertical velocity is oscillatory with non-negativity, and remains strictly above the initial height z_0 during the time interval.

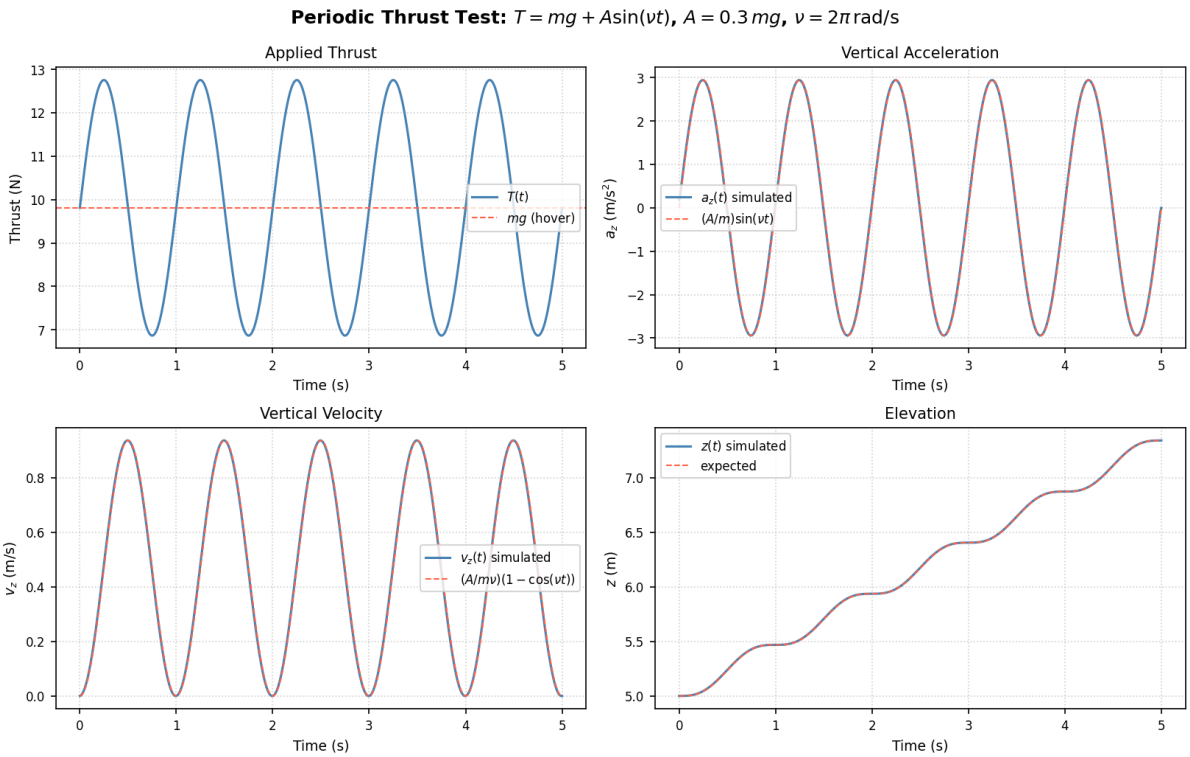

Figure 3: Vertical Dynamics' response under periodic thrust forcing.

For this experiment, we set $A = 0.3mg$, meaning that the maximum additional thrust applied is 30% above the hover thrust. From the applied thrust profile, we observe that the thrust oscillates about the hover value mg, with a maximum of approximately 12.5 N.

It is worth noting that the MN2212 T-motors (the proposed motors for the physical drone) are capable of producing up to 17.7 N of thrust per motor, giving a total maximum thrust of 70.8 N. In this experiment, the peak thrust corresponds to roughly 18% of the total available thrust. This is a reasonable operating regime, as it avoids excessive power usage and allows for longer flight times, rather than operating near maximum continuous thrust.

We set the angular frequency to $\nu = 2\pi$ rad/s, corresponding to one full oscillation per second.

The observed behaviour aligns with our theoretical and analytical expectations above. The vertical acceleration exhibits a sinusoidal profile, directly reflecting the oscillatory

thrust input. The vertical velocity is oscillatory but remains non-negative, with a minimum value of zero. Consequently, the altitude increases over time, with a smooth sinusoidal variation, and does not fall below the initial height.

We further explored the effect of varying A and ν independently. Increasing A while keeping ν fixed results in a greater gain in altitude, as expected from the increased magnitude of acceleration. In contrast, increasing ν while keeping A fixed leads to a smaller net increase in altitude. Intuitively, this occurs because the system spends less time at higher velocities before the acceleration reverses. Conversely, decreasing ν results in significantly larger altitude gains, since the drone maintains positive velocity for longer intervals.

Finally, when increasing ν to 100π, a numerical sampling issue arises. With a simulation timestep of $\Delta t = 0.01$, within `hover_env.py` we have:

$$\sin(100\pi \cdot 0.01) = \sin(\pi) = 0,$$

which causes the oscillatory input to be effectively lost in the discretisation. This is purely a numerical artefact. To resolve this, the timestep Δt must be reduced (such as say halved), allowing the simulation to properly capture the higher-frequency dynamics. We also note that, in practice, the physical system imposes additional constraints on how large the angular frequency ν can be. In particular, the Electronic Speed Controller (ESC), together with the motor dynamics and propeller spin-up delay, introduce a finite response time. As a result, the thrust cannot be adjusted arbitrarily fast, and thus this places a practical upper bound on the achievable angular frequency in real-world operation.

Experiment 4: Pure Roll Torque Test

To verify the rotational dynamics, we apply a non-zero roll torque while keeping the total thrust fixed at hover:

$$T = mg, \qquad \tau_x = \tau_0, \qquad \tau_y = \tau_z = 0.$$

Returning to the rotational equation of motion:

$$\dot{\omega}_x = \frac{\tau_x - (I_z - I_y)\omega_y\omega_z}{I_x},$$

we note that, since no torque is applied about the pitch or yaw axes, there is no induced

motion in those directions. Thus, we take:

$$\omega_y = \omega_z = 0,$$

which eliminates the coupling term. It follows that:

$$\dot{\omega}_x \approx \frac{\tau_x}{I_x}.$$

Hence, the angular acceleration is constant. Integrating with the initial condition $\omega_x(0) = 0$, we obtain:

$$\omega_x(t) = \frac{\tau_x}{I_x}\, t.$$

,

meaning that angular velocity evolves linearly in time.

Integrating once more, and applying the initial condition $\phi(0) = 0$, we obtain

$$\phi(t) = \frac{\tau_x}{2I_x}\, t^2,$$

which shows that the roll angle evolves quadratically in time.

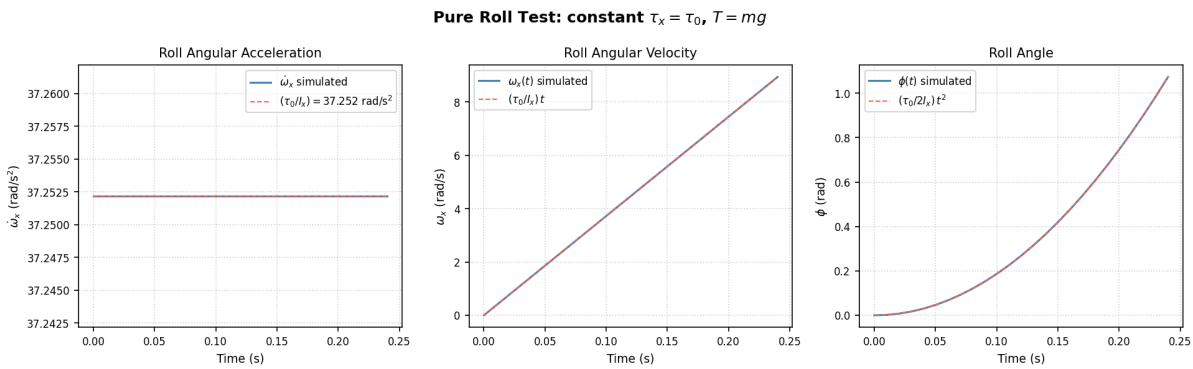

Figure 4: Roll response of drone under a pure roll torque input.

As expected, we observe a constant roll angular acceleration, a roll angular velocity that increases linearly with time, and a roll angle that increases quadratically with time.

We note that within the `hover_env.py` file, the `_is_terminated()` condition is triggered once the roll angle exceeds $\pi/3$. This condition is imposed for computational efficiency during training, and thus the simulation terminates just after the drone reaches this angle.

In practice, if we extend the objective to allow the drone to perform full rotations (i.e. flips), this termination condition would be removed or relaxed. Indeed, upon increasing the threshold to π radians, we confirm that the drone continues to roll smoothly until

reaching this larger roll angle.

Finally, we confirm that performing a pure pitch test yields an analogous set of results to the pure roll test: namely, constant angular acceleration, linearly increasing angular velocity, and a quadratically evolving pitch angle. The same behaviour is observed for the pure yaw test.

For the pure yaw case, we note that `hover_env.py` does not impose a constraint on the yaw angle. As a result, the simulation is not terminated early and instead runs for the full duration of `env.max_steps`, corresponding to approximately 5 seconds of simulation time.

Conclusion

We conclude that the non-linear quadrotor simulator accurately reflects the equations of motion derived from first principles. By comparing theoretical expectations with simulated responses under carefully chosen open-loop experiments, we verified that both the translational and rotational dynamics behave in a physically consistent and mathematically coherent manner.

This establishes a reliable foundation for the next stage of the project, namely the implementation and comparison of PID, LQR, and PPO controllers on the same verified non-linear system.

Appendix: System Parameters

Parameter	Value
Mass, m	1.0 kg
Gravity, g	$9.81\,\mathrm{m/s^2}$
Moment of inertia, I_x	0.005342 kg.m^2
Moment of inertia, I_y	0.005342 kg.m^2
Moment of inertia, I_z	0.008831 kg.m^2
Arm length, l	0.159 m
Torque-Thrust ratio, k_d	0.1
Time-step, Δt	0.01

Table 1: Physical and numerical parameters used in the simulator.

Code Availability

The implementation of the non-linear quadrotor simulator, including the dynamics model, numerical integration scheme, and experimental setup, is available in the following GitHub repository:

`https://github.com/HenryR888/robust-rl-quadrotor-control`

The repository ensures full reproducibility of the results presented in this report.

References

[1] Articulated Robotics, *Understanding the Rotation Matrix*, YouTube video. Available at: `https://www.youtube.com/watch?v=8GrdqAizcfU`. Accessed: 1 March 2026.

[2] Ross, S., *Euler Angle Rates & Angular Velocity – Kinematic Differential Equation for Rigid Body Dynamics*, YouTube video. Available at: `https://www.youtube.com/watch?v=Z8nwjouP58o`. Accessed: 1 March 2026.

[3] Beard, R. W. and McLain, T. W., *Small Unmanned Aircraft: Theory and Practice*, Princeton University Press, 2012.

[4] Sanca, A. S., Alsina, P. J., and Cerqueira, J. J. F., *Dynamic Modelling of a Quadrotor Aerial Vehicle with Nonlinear Inputs*.